

Database Management System

CASE STUDY 19: Tuition Class Batch and Fee Ledger

1. Title

Tuition Class Batch and Fee Ledger Database Using SQL

2. Problem Statement

A tuition centre runs subject batches at fixed timings, enrolls students into multiple batches, and collects fees in instalments.

The database must support the day-to-day working of the centre head and produce the reporting needed at the close of every month. It must answer questions such as: (1) How much fee is outstanding per batch? (2) Which students are enrolled in more than two batches? (3) Which batches are at full capacity? (4) How much was collected in instalments each month? (5) Can two counters enrol students into the last seat of a batch at the same time without overbooking?

3. Objectives

Students should:

- Model subjects, batches, faculty, enrolments and fee instalments so that outstanding fees per batch are always current.
- Implement one multi-step business operation as a single transaction using BEGIN, COMMIT, ROLLBACK and SAVEPOINT.
- Reproduce a concurrency problem from two sessions and prevent it using row-level locking and an appropriate isolation level.
- Using PostgreSQL, apply the core SQL covered in the course: filtering and sorting, aggregate functions with GROUP BY and HAVING, joins and subqueries.
- Seed the database with realistic data volume, not three rows per table.
- Record every design decision and the alternative that was rejected.

4. Outcomes

On completing this case study, the student:

- Explains each ACID property using evidence from their own database rather than a definition.
- Reproduces a lost update or dirty read scenario and describes the fix that closes it.
- Justifies the isolation level chosen for a given business operation.
- Defends this design against a reasonable alternative instead of asserting that it is correct.
- Explains why each query was written the way it was, and what it would cost to write it differently.



Database Management System

5. Deliverables

- ER diagram, as a PDF or an image.
- schema.sql containing all tables, keys and constraints.
- seed.sql containing realistic sample data.
- queries.sql containing every query written, with its output.
- README.md stating the design decisions taken and the reasoning behind them.
- Focus-specific deliverable: a two-session transcript showing the anomaly first and the fix afterwards.